



Getting Started with TypeScript

Learning Objectives

- Understand TypeScript purpose as a statically-typed superset of JavaScript and its advantages in code quality.
- Set up the development environment for TypeScript by installing necessary tools and configuring an editor.
- Use TypeScript basic types to enforce stricter type-checking in variables and function parameters.
- Learn to add type annotations to variables and functions to catch errors early and ensure consistent code behavior.
- Create custom types using interfaces and type aliases for better code readability and flexibility.

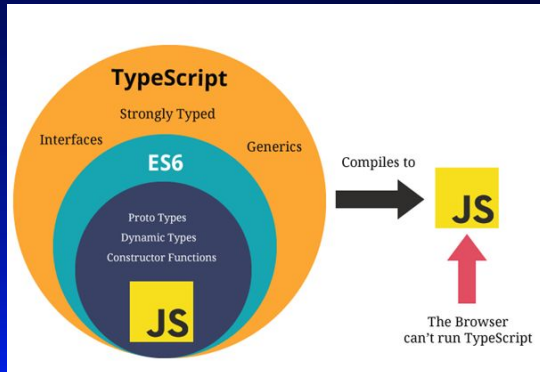




What is TypeScript?

What is TypeScript?

- TypeScript is a syntactic superset of JavaScript that introduces optional static typing and other features to enhance the development process.
- It was developed by Microsoft to improve the scalability and maintainability of large codebases.
- TypeScript allows developers to write more robust code by enabling type annotations, which help catch errors at compile time rather than at runtime



Static Typing in TypeScript: Explanation and Benefits

- Static typing in TypeScript allows developers to explicitly define the types of variables, function parameters, and return values.
- **This leads to several benefits:**
 - **Early Error Detection:** TypeScript checks types at compile time, helping to identify errors before the code is executed.
 - **Enhanced Code Clarity:** By specifying types, developers can better understand the expected data structures, making the code more readable and maintainable.
 - **Improved Tooling Support:** IDEs can provide better autocompletion and error-checking based on defined types.

```
1 // JavaScript dynamic typing
2 let x = 2;
3
4 // TypeScript static typing
5 let b: number = 31;
```

Static Typing in TypeScript: Explanation and Benefits

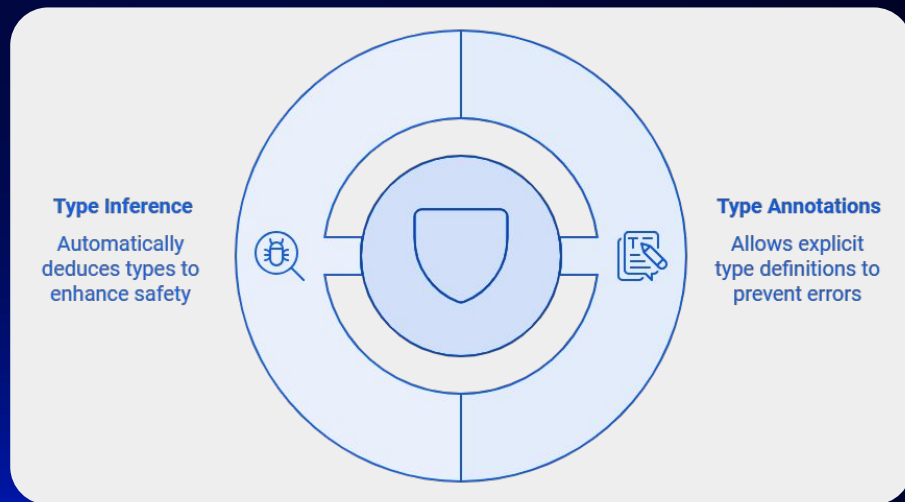


```
1 // JavaScript dynamic typing
2 let x = 2;
3
4 // TypeScript static typing
5 let b: number = 31;
```

Addressing Issues in Dynamically Typed Languages

- Dynamically typed languages like JavaScript can lead to runtime errors due to type mismatches.
- **TypeScript addresses these issues by:**
 - **Type Annotations:** Allowing developers to specify expected types, preventing accidental type errors.
 - **Type Inference:** Automatically inferring types based on assigned values, reducing the need for explicit type annotations while still maintaining type safety

Addressing Issues in Dynamically Typed Languages



Key Benefits of TypeScript

Enhanced Tooling

Providing advanced features for efficient coding



Early Error Detection

Identifying issues before runtime to prevent bugs



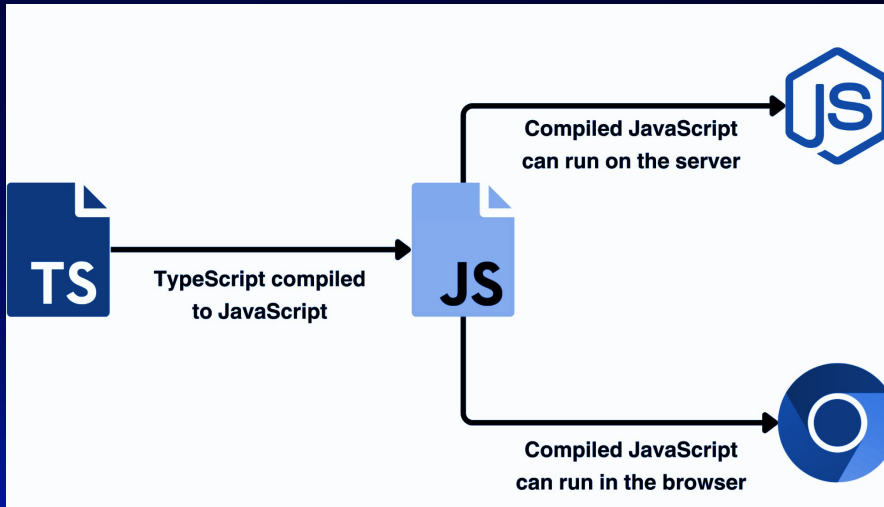
Code Readability

Making code easier to understand and maintain

TypeScript Compilation to JavaScript

- TypeScript code is not directly executable; it must be transpiled into standard JavaScript.
- **This process involves:**
 - **Transpilation:** The TypeScript compiler (tsc) converts TypeScript files (.ts) into JavaScript files (.js), stripping away type information while preserving the original functionality.
 - **Compatibility:** The resulting JavaScript can run in any environment that supports JavaScript, allowing developers to leverage existing libraries and frameworks seamlessly

TypeScript Compilation to JavaScript



Pop Quiz

Q.What is TypeScript

A

**A superset of JavaScript that
compiles to JavaScript**

B

**A framework for building
web applications**

Pop Quiz

Q.What is TypeScript

A

**A superset of JavaScript that
compiles to JavaScript**

B

**A framework for building
web applications**



Setting Up the Environment

Introduction to Node.js

- Node.js is a runtime environment for executing JavaScript code on the server side.
- It's event-driven, asynchronous, and single-threaded, making it efficient for handling concurrent tasks.
- Node.js is commonly used for building web servers, APIs, and real-time applications.
- It comes with npm, a package manager for easy library management.
- It's cross-platform, allowing developers to run their applications on various operating systems.

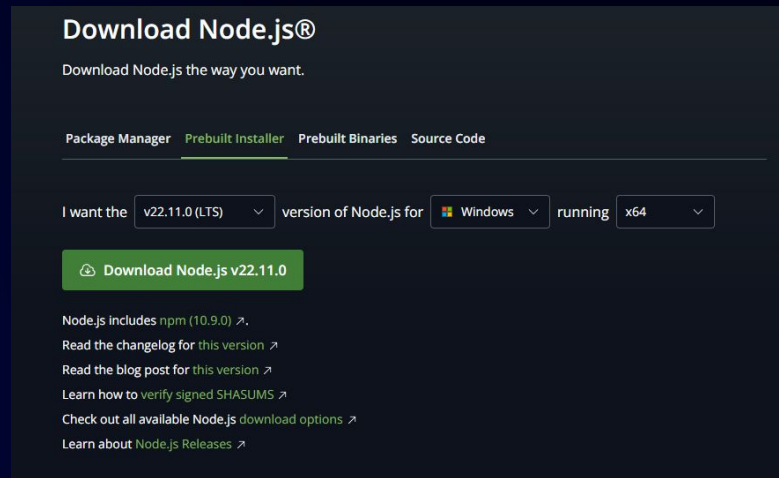


Key Features of Node.js



Installing Node.js

- Make sure you have Node Js in your system if not following the steps below –
 - **Download Node.js:** Visit the Node.js Downloads page and select the appropriate installer for your operating system (LTS version recommended).
[Link](#)
 - **Run Installer:** Launch the downloaded installer, accept the license agreement, and follow the prompts to complete the installation



Installing TypeScript Globally

- Open Terminal/Command Prompt: Ensure you have administrative privileges.
- Run Installation Command: to install TypeScript globally using npm (Node Package Manager).

npm install typescript -g



Setting Up Visual Studio Code (VSCode) for TypeScript

- **Download VSCode:** Visit the Visual Studio Code website and download the installer for your OS.
- **Install VSCode:** Follow the installation instructions to set up VSCode on your machine.
- **Install TypeScript Extension:** Open VSCode, go to Extensions (Ctrl+Shift+X), and search for "TypeScript" to install relevant extensions for enhanced support.

Setting Up Visual Studio Code (VSCode) for TypeScript

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows
Windows 10, 11

User Installer [x64](#) [Arm64](#)
System Installer [x64](#) [Arm64](#)
Installer .zip [x64](#) [Arm64](#)
CLI [x64](#) [Arm64](#)



↓ .deb
Debian, Ubuntu

↓ .rpm
Red Hat, Fedora, SUSE

.deb [x64](#) [Arm32](#) [Arm64](#)
.rpm [x64](#) [Arm32](#) [Arm64](#)
.tar.gz [x64](#) [Arm32](#) [Arm64](#)
Snap [Snap Store](#)
CLI [x64](#) [Arm32](#) [Arm64](#)



↓ Mac
macOS 10.15+

.zip [Intel chip](#) [Apple silicon](#) [Universal](#)
CLI [Intel chip](#) [Apple silicon](#)



JavaScript and TypeScript Nightly

Microsoft [microsoft.com](#) |  4,356,321 installs |      (49) | Free

Enables typescript@next to power VS Code's built-in JavaScript and TypeScript support

[Install](#) [Trouble Installing?](#)

Verifying the Installation

- Run the following command to verify the installation-
 - Nodejs: **node --v**
 - Typescript: **tsc -v**

```
Ineuron@mang-dev MINGW64 ~  
●  node -v  
○ v20.14.0
```

```
Ineuron@mang-dev MINGW64 ~  
●  tsc -v  
○ Version 5.1.6
```

Configure TypeScript with Nodejs Project

- Once TypeScript is installed, you'll need to create and configure the **tsconfig.json** file, which is essential for defining compiler options and project settings.
- To generate a basic tsconfig.json file, execute the command:

npx tsc --init

Configure TypeScript with Nodejs Project

```
Ineuron@mang-dev MINGW64 /d/Mang/typescript
```

```
• npx tsc --init
```

```
Created a new tsconfig.json with:
```

```
target: es2016
module: commonjs
strict: true
esModuleInterop: true
skipLibCheck: true
forceConsistentCasingInFileNames: true
```

```
You can learn more at https://aka.ms/tsconfig
```

TS

Note – will have more understanding on the tsconfig.json in the upcoming chapter



Hands on demo || Setting Up the Environment

Pop Quiz

Q. What is the purpose of the tsconfig.json file?

A

It configures project dependencies.

B

**It specifies compiler options and settings
for the TypeScript project.**

Pop Quiz

Q. What is the purpose of the tsconfig.json file?

A

It configures project dependencies.

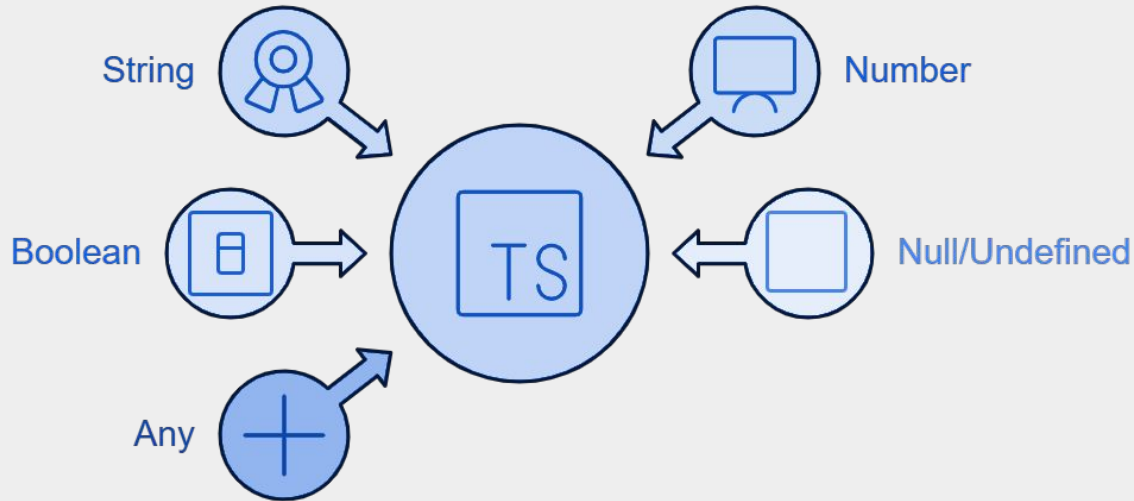
B

**It specifies compiler options and settings
for the TypeScript project.**



Basic Types

Overview of Basic Types



Type Examples and Definitions

- **String:**
 - Represents text data
 - e.g., **let name: string = "Alice";**
- **Number:**
 - Represents both integer and floating-point numbers
 - e.g., **let name: string = "Alice";**
- **Boolean:**
 - Represents true/false values
 - e.g., **let isActive: boolean = true;**
- **Null:**
 - A special type that represents the absence of a value.
 - e.g., **let nullableValue: null = null;**

Type Examples and Definitions

- **Undefined:**
 - A type that represents an uninitialized variable or the lack of a value.
 - e.g., `let uninitializedValue: undefined = undefined;`
- **Any:**
 - A flexible type that can hold any value.
 - e.g.,
`let randomValue: any = "Hello";`
`randomValue = 123; // valid, but should be used sparingly`
- **Array:**
 - Represents a collection of values, all of the same type.
 - e.g., `let numbers: number[] = [1, 2, 3]`
- **Tuple:**
 - Represents an array with fixed types and a fixed number of element
 - e.g., `let tuple: [string, number] = ["Bob", 25];`

Why Use Strict Typing?

Collaboration

Enhances team communication with clear data structure expectations.



Error Prevention

Catches type-related errors at compile-time, reducing bugs.



Improved Refactoring

Static typing aids in safely modifying code without errors.



Code Clarity

Explicit types serve as documentation, enhancing understanding.



Practical Examples of Strict Typing Benefits

```
function greet(person: { name: string }) {  
    return "Hello, " + person.name;  
}  
  
greet({ name: "Alice" }); // Correct usage  
greet({ name: "Alice", age: 30 }); // Error if strict typing is enforced
```




Hands on demo || Basic Types

Pop Quiz

Q. Which of these types is used in TypeScript for a variable that may have no assigned value?

A

undefined

B

void

Pop Quiz

Q. Which of these types is used in TypeScript for a variable that may have no assigned value?

A

undefined

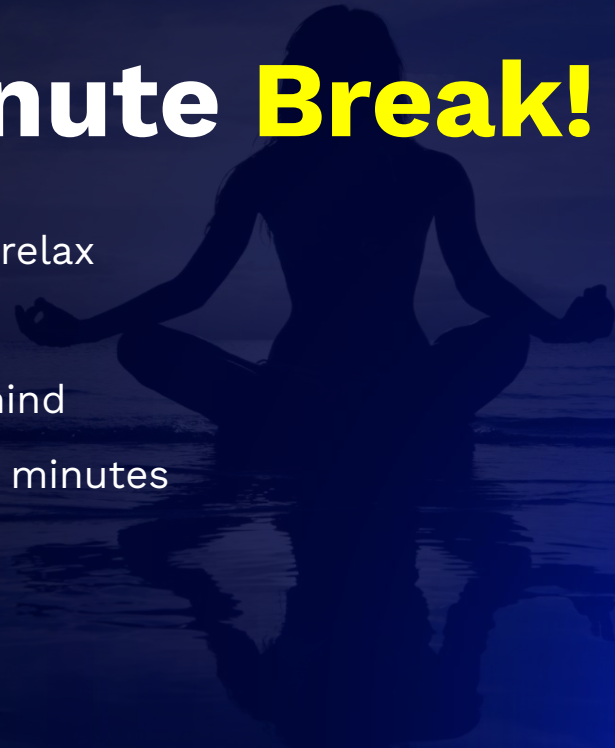
B

void



Take A 5 -Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

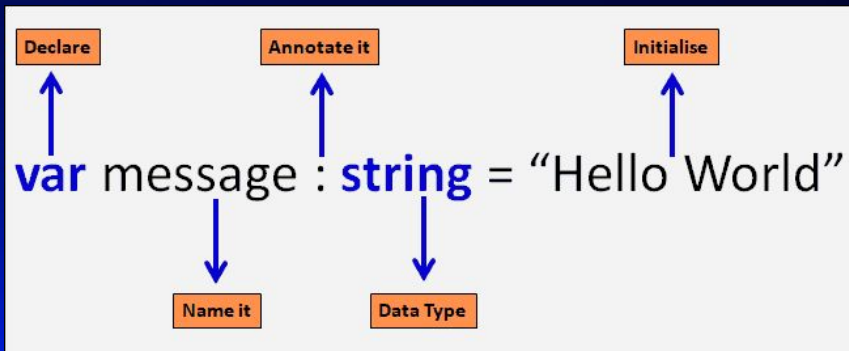




Type Annotations

Introduction to Type Annotations

- Type annotations allow developers to explicitly specify the types of variables, function parameters, and return values in TypeScript.
- Enhances code readability, helps with early error detection, and improves maintainability by providing clear expectations for data types.



Annotating Types for Different Scenarios

- Variables
 - example -

```
let age: number = 25;  
let name: string = "Alice";
```

- Function Parameters
 - example -

```
function greet(person: string) {  
    return "Hello, " + person;  
}
```

Annotating Types for Different Scenarios

- Function Return types
 - example -

```
function greet(person: string): string {  
    return "Hello, " + person;  
}
```


Type Enforcement in TypeScript

- **Static Typing:** TypeScript enforces type checks at compile time, ensuring that variables and functions adhere to defined types.
- **Error Detection:** If a variable is assigned a value of an incorrect type, TypeScript will throw a compilation error.
- **Index Signatures:** Allows enforcing types for object properties dynamically



Static Typing



Error
Detection



Index
Signatures

Type Inference

- TypeScript can automatically infer the type of a variable based on its assigned value without explicit annotations.
- where TypeScript can automatically detect the type based on the initial value, reducing the need for explicit annotations.
- **Benefits –**
 - Reduces verbosity while maintaining type safety.
 - Allows developers to write cleaner code without losing the advantages of static typing.

```
// example  
let count = 10; // inferred as number  
let greeting = "Hello"; // inferred as string
```



Hands on Demo || Type Annotations

Pop Quiz

Q. Which of the following correctly applies a type annotation to a variable in TypeScript?

A

```
let age: number = 30;
```

B

```
let age = number: 30;
```

Pop Quiz

Q. Which of the following correctly applies a type annotation to a variable in TypeScript?

A

```
let age: number = 30;
```

B

```
let age = number: 30;
```



Interfaces and Type Aliases

What are Interfaces?

- Interfaces are a way to define the structure of an object in TypeScript, specifying the properties and their types.
- They enable type-checking and help ensure that objects conform to a specific shape, enhancing code reliability and readability.

```
// Example
interface Person {
  name: string;
  age: number;
}
```

Using Interfaces in Functions

- **Function Parameters:** Interfaces can be used to define the shape of objects passed to functions.

Example:

```
interface Person {  
  name: string;  
  age: number;  
}
```

```
// functions  
function greet(person: Person): string {  
  return `Hello, ${person.name}`;  
}  
  
const user: Person = { name: "Alice", age: 30 };  
console.log(greet(user)); // Output: Hello, Alice
```


What are Type Aliases?

- Type aliases allow developers to create a new name for a type, which can be used for primitives, unions, intersections, tuples, and more.
- They provide flexibility in defining types and can simplify complex type definitions.

```
// Example  
type ID = number;
```

Syntax and Usage of Type Aliases

```
// Basic syntax
type User = {
  id: ID;
  name: string;
  isActive: boolean;
};

const user1: User = { id: "123", name: "Bob", isActive: true };
```

```
// Union Types
type StringOrNumber = string | number;
```

Interfaces vs. Type Aliases: Key Differences

Feature	Interface	Type Alias
Definition	Defines the shape of an object or structure.	Provides a new name for a type, which can include complex types.
Usage	Primarily used for objects and classes.	Can define objects, primitive types, unions, tuples, etc.
Extending	Supports extending other interfaces using extends.	Can create new types by combining other types using intersection (&) and union (

Interfaces vs. Type Aliases: Key Differences

Preferred For	Structuring objects and classes, especially when properties may be extended later.	Defining unions, intersections, and primitives, or when flexibility in types is required.
Implementation in Classes	Can be implemented in classes to enforce shape compliance.	Cannot be implemented directly in classes, but can be used within them as types.

Code Examples of Interfaces and Type Aliases

```
// Interfaces example
interface Animal {
    species: string;
    sound(): void;
}

class Cat implements Animal {
    species = "Cat";
    sound() {
        console.log("Meow!");
    }
}

const cat = new Cat();
cat.sound(); // Output: Meow!
```

```
// Type aliases example
type Point = {
    x: number;
    y: number;
};

const pointA: Point = { x: 10, y:
20 };

type Shape = Point & { color?:
string }; // Intersection type
const shapeA: Shape = { x: 5, y:
15, color: "red" };
```



Hands on demo || Interfaces and Type Aliases

Pop Quiz

Q. In TypeScript, which keyword is used to define a type alias?

A

alias

B

type

Pop Quiz

Q. In TypeScript, which keyword is used to define a type alias?

A

alias

B

type



Best Practices – Quick Roundup

Avoid Using any Type

- The **any** type allows any value, effectively opting out of TypeScript type system.
- It is recommended to avoid using **any** as it defeats the purpose of type safety and can lead to runtime errors.
- Alternatively use specific types or union types to maintain type safety and clarity in your code.



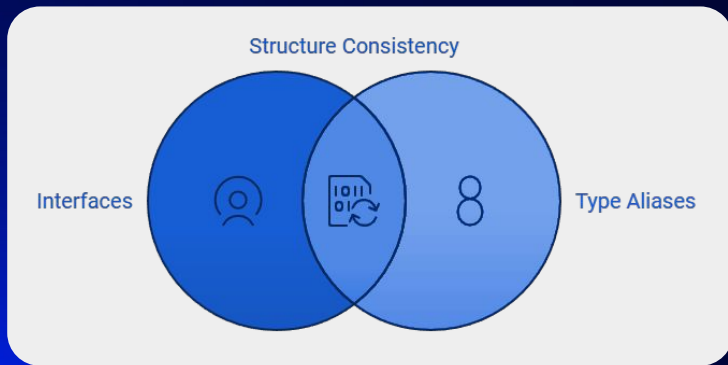
Use Type Annotations Consistently

- It is important to consistently use type annotations as it improves code readability and maintainability.
- It is best to always annotate function parameters, return types, and complex object types.



Define Interfaces or Type Aliases for Structure Consistency

- Interfaces and type aliases help define the structure of objects and ensure consistency across your codebase.
- It is recommended to use interfaces for defining object shapes and type aliases for unions or complex types.



Choose Interfaces for Object Shapes; Type Aliases for Unions

- **Interfaces:**

- Best for defining the shape of objects and supporting inheritance.
- Can be extended or merged, providing flexibility.

- **Type Aliases:**

- Ideal for creating unions, intersections, or more complex types.
- Cannot be merged but can represent more complex structures.



Interfaces

Shape and inheritance



Type Aliases

Complex structures and unions

Pop Quiz

Q. Which of the following is a best practice when working with any in TypeScript?

A

Use any as frequently as possible.

B

Use any only when necessary and with caution.

Pop Quiz

Q. Which of the following is a best practice when working with any in TypeScript?

A

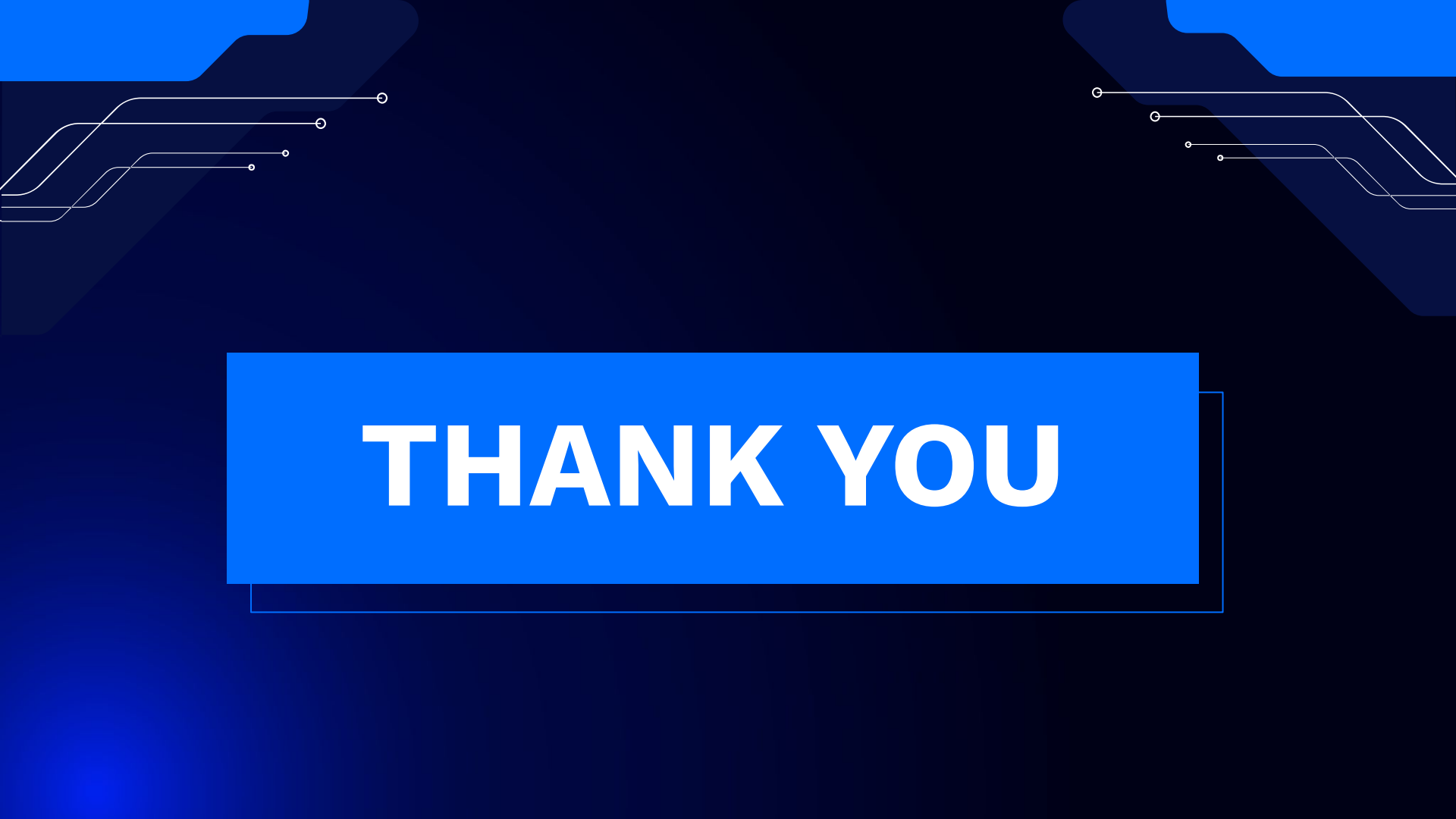
Use any as frequently as possible.

B

Use any only when necessary and with caution.

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU